

COMPRAR BEM - SISTEMA DE GESTÃO DE COMPRAS PÚBLICAS

Extratos de Código-Fonte para Registro INPI

INFORMAÇÕES DO SOFTWARE

Nome do Software: Comprar Bem

Versão: 0.1.0

Descrição: Sistema de gestão de compras e licitações públicas com funcionalidades de:

- Pré-qualificação e padronização de produtos (DCB)
- Catálogo eletrônico de produtos padronizados
- Pesquisa de mercado e referência de preços
- Gestão de processos administrativos
- Avaliação de desempenho pós-compra (RDM)
- Assistente de IA especializado em licitações (QualiBot 2.0)

Tecnologias: Vue.js 2.7.16, Node.js, Supabase (PostgreSQL), Google Gemini AI

Data: Dezembro de 2025

1. ARQUITETURA PRINCIPAL DO SISTEMA

1.1 Configuração do Projeto (package.json)

Arquivo: package.json

```
{
  "name": "comprar-bem",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "serve": "vue-cli-service serve",
    "build": "vue-cli-service build",
    "lint": "vue-cli-service lint",
    "deploy": "vue-cli-service build && gh-pages -d dist",
    "test": "jest"
  },
  "dependencies": {
    "@emailjs/browser": "^4.4.1",
    "@fortawesome/fontawesome-free": "^6.7.2",
    "@google/generative-ai": "^0.2.1",
    "@supabase/supabase-js": "^2.49.4",
    "axios": "^1.10.0",
    "html2pdf.js": "^0.10.3",
    "jspdf": "^3.0.1",
    "qrcode": "^1.5.4",
    "vue": "^2.7.16",
    "vue-router": "^3.6.5",
    "vue-sweetalert2": "^5.0.11",
    "vuex": "^3.6.2"
  }
}
```

1.2 Inicialização da Aplicação (main.js)

Arquivo: src/main.js

```
import Vue from 'vue'
import App from './App.vue'
import router from './router'
import store from './store'
import VueSweetalert2 from 'vue-sweetalert2'
import 'sweetalert2/dist/sweetalert2.min.css'
import Loading from 'vue-loading-overlay'
import 'vue-loading-overlay/dist/vue-loading.css'
import '@fortawesome/fontawesome-free/css/all.css'

Vue.config.productionTip = false

Vue.use(VueSweetalert2)
Vue.use(Loading)

new Vue({
  router,
  store,
  render: h => h(App),
  mounted() {
    console.log('🚀 Sistema Comprar Bem carregado')
  }
}).$mount('#app')
```

1.3 Gerenciamento de Estado Global (store/index.js)

Arquivo: src/store/index.js

```
import Vue from 'vue'
import Vuex from 'vuex'
import { supabase } from '@/services/supabase'

Vue.use(Vuex)

export default new Vuex.Store({
  state: {
    user: null,
    tenant: null,
    isLoading: false
  },
  mutations: {
    SET_USER(state, user) {
      state.user = user
    },
    SET_TENANT(state, tenant) {
      state.tenant = tenant
    },
    SET_LOADING(state, isLoading) {
      state.isLoading = isLoading
    }
  },
  actions: {
    async fetchUser({ commit }) {
      const { data } = await supabase.auth.getSession()
      const user = data?.session?.user
      commit('SET_USER', user)

      if (user) {
        const { data: tenantData } = await supabase
          .from('tenants')
          .select('*')
          .eq('id', user.user_metadata?.tenant_id)
          .single()

        if (tenantData) {
          commit('SET_TENANT', tenantData)
        }
      }
    },
  },
  getters: {
    isAuthenticated: state => !!state.user,
    currentUser: state => state.user,
    currentTenant: state => state.tenant
  }
})
```

2. SISTEMA DE ROTEAMENTO E CONTROLE DE ACESSO

2.1 Configuração de Rotas (router/index.js) - Parte 1

Arquivo: src/router/index.js (linhas 1-100)

```
import Vue from 'vue'
import VueRouter from 'vue-router'
import Home from '../views/Home.vue'
import Dashboard from '../views/Dashboard.vue'
import Cadastro from '../views/Cadastro.vue'
import Analise from '../views/Analise.vue'
import Catalogo from '../views/Catalogo.vue'
import Feedback from '../views/Feedback.vue'
import DCB from '../views/DCB.vue'
import CCL from '../views/CCL.vue'
import Classificacao from '../views/Classificacao.vue'
import ProcessosAdministrativos from '../views/ProcessosAdministrativos.vue'

import { supabase } from '../services/supabase'
import PerfilUsuarioService from '../services/perfilUsuarioService'

Vue.use(VueRouter)

const routes = [
  { path: '/', name: 'Home', component: Home, meta: { requiresAuth: false } },
  {
    path: '/dashboard',
    name: 'Dashboard',
    component: Dashboard,
    meta: {
      requiresAuth: true,
      requiredProfiles: ['cpm'],
      moduloRequired: 'dashboard'
    }
  },
  {
    path: '/dcb',
    name: 'DCB',
    component: DCB,
    meta: {
      requiresAuth: true,
      requiredProfiles: ['cpm'],
      moduloRequired: 'dcb'
    }
  },
  {
    path: '/processos-administrativos',
    name: 'ProcessosAdministrativos',
    component: ProcessosAdministrativos,
    meta: {
      requiresAuth: true,
      requiredProfiles: ['cpm', 'orgao_administrativo', 'assessoria_juridica'],
      moduloRequired: 'processos_administrativos'
    }
  }
]

const router = new VueRouter({
  mode: process.env.NODE_ENV === 'production' ? 'hash' : 'history',
  base: process.env.BASE_URL,
  routes
})
```

2.2 Sistema de Autenticação e Autorização (router/index.js) - Parte 2

Arquivo: src/router/index.js (linhas 204-297)

```
// Guard de navegação para verificar autenticação e perfis
router.beforeEach(async (to, from, next) => {
  const requiresAuth = to.matched.some(record => record.meta.requiresAuth)

  if (!requiresAuth) {
    next()
    return
  }

  try {
    const { data } = await supabase.auth.getSession()
    const user = data?.session?.user

    if (requiresAuth && !user) {
      next('/')
      return
    }

    const requiredProfiles = to.meta.requiredProfiles
    const moduloRequired = to.meta.moduloRequired

    if (requiredProfiles || moduloRequired) {
      try {
        const perfilUsuario = await PerfilUsuarioService.obterPerfilUsuarioAtual()

        if (!perfilUsuario) {
          next('/acesso-negado')
          return
        }

        // CCL: só pode acessar painel CCL
        if (perfilUsuario.perfil_usuario === 'ccl' && to.path !== '/ccl') {
          next('/ccl')
          return
        }

        // Verificar perfil permitido
        if (requiredProfiles && !requiredProfiles.includes(perfilUsuario.perfil_usuario)) {
          next('/acesso-negado')
          return
        }

        // Verificar acesso ao módulo
        if (moduloRequired) {
          const temAcesso = await PerfilUsuarioService.verificarAcessoModulo(moduloRequired)
          if (!temAcesso) {
            next('/acesso-negado')
            return
          }
        }

        next()
      } catch (error) {
        console.error('Erro ao verificar permissões:', error)
        next('/acesso-negado')
      }
    } else {
      next()
    }
  }
})
```

```
    }  
  } catch (error) {  
    console.error('Erro na verificação de autenticação:', error)  
    next('/')  
  }  
})
```

3. CAMADA DE INTEGRAÇÃO COM BANCO DE DADOS

3.1 Configuração Supabase (services/supabase.js)

Arquivo: `src/services/supabase.js`

```
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = 'https://ruagsbbczuwgfflgcaol.supabase.co'
const supabaseAnonKey = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9... '

export const supabase = createClient(supabaseUrl, supabaseAnonKey)

// Função para obter tenant_id do usuário logado
export const getTenantId = async () => {
  const { data } = await supabase.auth.getSession()
  const user = data?.session?.user
  return user?.user_metadata?.tenant_id || null
}

export default supabase
```

4. SISTEMA DE INTELIGÊNCIA ARTIFICIAL - QUALIBOT 2.0

4.1 Serviço Gemini AI com Fallback (services/geminiService.js) - Parte 1

Arquivo: src/services/geminiService.js (linhas 1-90)

```
const GEMINI_API_KEY = 'AIzaSyCm9ku5Y5oonPZ5c-LgX4AHLIyK_7I3deY';

// Lista de modelos em ordem de prioridade
const GEMINI_MODELS = [
  {
    name: 'gemini-2.0-flash-exp',
    url: 'https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash-exp:generate',
    description: 'Gemini 2.0 Flash (Mais Novo e Rápido - Experimental)'
  },
  {
    name: 'gemini-1.5-flash',
    url: 'https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent',
    description: 'Gemini 1.5 Flash (Rápido e Eficiente)'
  },
  {
    name: 'gemini-1.5-pro',
    url: 'https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-pro:generateContent',
    description: 'Gemini 1.5 Pro (Mais Avançado)'
  },
  {
    name: 'gemini-1.0-pro',
    url: 'https://generativelanguage.googleapis.com/v1beta/models/gemini-1.0-pro:generateContent',
    description: 'Gemini 1.0 Pro (Versão Estável)'
  }
];

// Configurações de retry com exponential backoff
const RETRY_CONFIG = {
  maxRetries: 3,
  baseDelay: 800,
  maxDelay: 12000,
  backoffFactor: 1.8
};

const sleep = (ms) => new Promise(resolve => setTimeout(resolve, ms));

const calculateDelay = (attempt) => {
  const delay = RETRY_CONFIG.baseDelay * Math.pow(RETRY_CONFIG.backoffFactor, attempt);
  return Math.min(delay, RETRY_CONFIG.maxDelay);
};

// Verificar se erro é temporário
const isRetryableError = (error, response) => {
  if (error && error.message && error.message.includes('Failed to fetch')) {
    return true;
  }

  if (response) {
    const status = response.status;
    return status === 429 || status === 503 || status === 502 || status === 504;
  }

  return false;
};

// Verificar se deve tentar outro modelo
const shouldTryNextModel = (error, response) => {
  if (isRetryableError(error, response)) {

```

```
        return false;
    }

    if (response) {
        const status = response.status;
        return status === 400 || status === 401 || status === 403 ||
            status === 404 || status === 500;
    }

    return true;
};

const safeJsonParse = (text) => {
    try {
        return JSON.parse(text);
    } catch (e) {
        console.error('Erro ao fazer parse do JSON:', e);
        return null;
    }
};
```

4.2 Lógica de Tentativa com Retry (services/geminiService.js) - Parte 2

Arquivo: src/services/geminiService.js (linhas 90-201)

```
// Função para tentar um modelo específico
const tryModel = async (model, message, attempt = 0) => {
  try {
    const apiUrl = `${model.url}?key=${GEMINI_API_KEY}`;

    console.log(`Tentando modelo ${model.name} (tentativa ${attempt + 1})`);

    const response = await fetch(apiUrl, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
        'Accept': 'application/json'
      },
      body: JSON.stringify({
        contents: [{
          parts: [{
            text: `Você é o QualiBot 2.0, um assistente especializado em
licitações e compras públicas. Responda: ${message}`
          }]
        }]
      })
    });

    // Verificar resposta
    if (!response.ok) {
      const isTemporaryError = isRetryableError(null, response);

      if (isTemporaryError && attempt < RETRY_CONFIG.maxRetries) {
        const delay = calculateDelay(attempt);
        console.log(`Erro temporário. Aguardando ${delay}ms...`);
        await sleep(delay);
        return tryModel(model, message, attempt + 1);
      }

      throw new Error(`Modelo ${model.name} falhou: ${response.status}`);
    }

    const responseText = await response.text();
    const data = safeJsonParse(responseText);

    if (!data || !data.candidates?.[0]?.content?.parts?.[0]?.text) {
      if (attempt < RETRY_CONFIG.maxRetries) {
        await sleep(calculateDelay(attempt));
        return tryModel(model, message, attempt + 1);
      }
      throw new Error('Resposta inválida');
    }

    console.log(`✅ Sucesso com modelo ${model.name}!`);
    return {
      text: data.candidates[0].content.parts[0].text,
      modelUsed: model.name
    };
  } catch (error) {
    if (isRetryableError(error, null) && attempt < RETRY_CONFIG.maxRetries) {
      await sleep(calculateDelay(attempt));
      return tryModel(model, message, attempt + 1);
    }
  }
}
```

```
    }  
    throw error;  
  }  
};
```

4.3 Sistema de Fallback entre Modelos (services/geminiService.js) - Parte 3

Arquivo: src/services/geminiService.js (linhas 203-253)

```
export const geminiService = {
  async chat(message) {
    let lastError = null;
    let modelsAttempted = [];

    // Tentar cada modelo em ordem de prioridade
    for (let i = 0; i < GEMINI_MODELS.length; i++) {
      const model = GEMINI_MODELS[i];
      modelsAttempted.push(model.name);

      try {
        console.log(`🚀 Tentando modelo ${i + 1}/${GEMINI_MODELS.length}: ${model.name}`);
        const result = await tryModel(model, message);

        console.log(`✅ Resposta obtida com: ${result.modelUsed}`);
        return result.text;
      } catch (error) {
        lastError = error;
        console.error(`❌ Modelo ${model.name} falhou:`, error.message);

        // Se não é o último modelo, tentar o próximo
        if (i < GEMINI_MODELS.length - 1) {
          console.log(`🔄 Tentando próximo modelo: ${GEMINI_MODELS[i + 1].name}`);
          continue;
        }
      }
    }

    // Todos os modelos falharam
    console.error(`❌ Todos os modelos falharam:`, modelsAttempted);

    if (lastError?.message?.includes('Failed to fetch')) {
      throw new Error('Não foi possível conectar aos serviços de IA.');
```

5. SISTEMA DE GESTÃO DE PROCESSOS ADMINISTRATIVOS

5.1 Serviço de Processos (services/processosAdministrativosService.js)

Arquivo: src/services/processosAdministrativosService.js (linhas 1-96)

```
import { supabase, getTenantId } from './supabase'
import EmailNotificationService from './emailNotificationService'
import NumeracaoDocumentosService from './numeracaoDocumentosService'

export class ProcessosAdministrativosService {

  // Listar processos com filtros
  static async listarProcessos(filtros = {}) {
    try {
      const tenantId = await getTenantId()
      if (!tenantId) {
        throw new Error('Usuário não autenticado ou sem tenant')
      }

      let query = supabase
        .from('vw_processos_resumo')
        .select('*')
        .eq('tenant_id', tenantId)
        .order('data_autuacao', { ascending: false })

      // Aplicar filtros
      if (filtros.tipo_processo) {
        query = query.eq('tipo_processo', filtros.tipo_processo)
      }

      if (filtros.status) {
        query = query.eq('status', filtros.status)
      }

      if (filtros.busca) {
        query = query.or(`numero_processo.ilike.${filtros.busca}%,objeto.ilike.${filtros.busca}%`)
      }

      const { data, error } = await query

      if (error) throw error
      return data || []
    } catch (error) {
      console.error('Erro ao listar processos:', error)
      throw error
    }
  }

  // Obter processo completo
  static async obterProcesso(id) {
    try {
      const tenantId = await getTenantId()
      if (!tenantId) {
        throw new Error('Usuário não autenticado')
      }

      const { data, error } = await supabase
        .from('processos_administrativos')
        .select(`
          *,
          documentos:documentos_processo(*),
          produtos:produtos_prequalificacao(*),
          recursos:recursos_processo(*),
        `)
    }
  }
}
```

```
        dfd:dfd_processo(*),
        tramitacao:tramitacao_processo(*)
    `)
    .eq('id', id)
    .eq('tenant_id', tenantId)
    .single()

    if (error) throw error

    // Calcular totais
    if (data) {
        data.total_documentos = data.documentos?.length || 0
        data.total_produtos = data.produtos?.length || 0
    }

    return data
} catch (error) {
    console.error('Erro ao obter processo:', error)
    throw error
}
}
```

6. SISTEMA DE GESTÃO DE PRODUTOS E NOTIFICAÇÕES RDM

6.1 Serviço de Produtos RDM (services/produtoService.js)

Arquivo: src/services/produtoService.js (linhas 1-100)

```
import { supabase } from './supabase'
import { enviarEmailNovoProduto } from './emailService'

// Adicionar produto a usuário RDM
export async function adicionarProdutoUsuarioRDM(produtoData) {
  try {
    console.log('📦 Adicionando produto ao usuário RDM:', produtoData)

    const {
      tenant_id,
      usuario_rdm_id,
      produto_id,
      nome_produto,
      codigo_produto,
      descricao_produto,
      categoria_produto,
      data_recebimento = null
    } = produtoData

    // Validações
    if (!tenant_id || !usuario_rdm_id || !produto_id) {
      throw new Error('Dados obrigatórios não fornecidos')
    }

    // Buscar usuário RDM
    const { data: usuarioRDM, error: usuarioError } = await supabase
      .from('usuarios_rdm')
      .select('*')
      .eq('id', usuario_rdm_id)
      .eq('tenant_id', tenant_id)
      .single()

    if (usuarioError || !usuarioRDM) {
      throw new Error('Usuário RDM não encontrado')
    }

    // Adicionar produto ao usuário
    const { data: vinculo, error: vinculoError } = await supabase
      .rpc('adicionar_produto_usuario_rdm', {
        p_tenant_id: tenant_id,
        p_usuario_rdm_id: usuario_rdm_id,
        p_produto_id: produto_id,
        p_data_recebimento: data_recebimento
      })

    if (vinculoError) {
      throw new Error('Erro ao vincular produto: ' + vinculoError.message)
    }

    console.log('✅ Produto vinculado com sucesso!')

    // Enviar notificação por email
    const dadosNotificacao = {
      produto_id,
      usuario_rdm_id,
      usuario_nome: usuarioRDM.nome_usuario,
      usuario_email: usuarioRDM.email,
      nome_produto: nome_produto || 'Produto Aprovado',
    }
  }
}
```

```
    codigo_produto: codigo_produto || `PROD-${produto_id}`,
    descricao_produto,
    categoria_produto,
    unidade_setor: usuarioRDM.unidade_setor,
    data_sincronizacao: new Date().toLocaleDateString('pt-BR')
  }

  try {
    console.log('📧 Enviando notificação...')
    const resultado = await enviarEmailNovoProduto(dadosNotificacao)

    if (resultado.success) {
      console.log('✅ Email enviado com sucesso!')

      // Marcar como notificado
      await supabase
        .from('usuarios_rdm_produtos')
        .update({
          notificado_email: true,
          data_notificacao: new Date().toISOString()
        })
        .eq('usuario_rdm_id', usuario_rdm_id)
        .eq('produto_id', produto_id)
    }
  } catch (emailError) {
    console.warn('⚠️ Erro ao enviar email:', emailError)
  }

  return {
    success: true,
    message: 'Produto adicionado com sucesso!',
    vinculo_id: vinculo
  }

} catch (error) {
  console.error('Erro ao adicionar produto:', error)
  throw error
}
}
```

7. COMPONENTES DE INTERFACE - QUALIBOT 2.0

7.1 Componente AIChat (components/common/AIChat.vue) - Template

Arquivo: src/components/common/AIChat.vue (linhas 1-88)

```
<template>
<div class="ai-chat" :class="{ 'ai-chat--expanded': isExpanded }">
  <!-- Botão flutuante -->
  <button
    v-if="!isExpanded"
    class="ai-chat__button"
    @click="toggleChat"
    title="QualiBot 2.0 - Assistente de Licitações"
  >
    <div class="ai-chat__button-content">
      <i class="fas fa-robot"></i>
      <span class="ai-chat__button-label">QualiBot 2.0</span>
    </div>
    <div class="ai-chat__notification" v-if="!hasInteracted">
      <i class="fas fa-bolt"></i> Como posso ajudar?
    </div>
  </button>

  <!-- Janela do chat -->
  <div v-else class="ai-chat__window">
    <div class="ai-chat__header">
      <div class="ai-chat__header-info">
        <div class="ai-chat__logo">
          <i class="fas fa-robot"></i>
          <i class="fas fa-bolt"></i>
        </div>
        <div>
          <h3>QualiBot 2.0</h3>
          <span class="ai-chat__status">
            <span class="ai-chat__status-dot"></span>
            Online e Pronto
          </span>
        </div>
      </div>
      <button class="ai-chat__close" @click="toggleChat">
        <i class="fas fa-times"></i>
      </button>
    </div>

    <div class="ai-chat__messages" ref="messagesContainer">
      <div
        v-for="(message, index) in messages"
        :key="index"
        :class="['ai-chat__message', `ai-chat__message--${message.type}`]"
      >
        <div class="ai-chat__message-avatar" v-if="message.type === 'assistant'">
          <i class="fas fa-robot"></i>
        </div>
        <div class="ai-chat__message-content">
          <span>{{ message.text }}</span>
        </div>
      </div>
    </div>

    <div class="ai-chat__input">
      <textarea
        v-model="currentMessage"
        @keydown.enter.prevent="sendMessage"
      >
    </div>
  </div>
</div>
```

```
        placeholder="Digite sua dúvida sobre licitações..."
        rows="2"
    ></textarea>
    <button
      @click="sendMessage"
      :disabled="isLoading || !currentMessage.trim()"
      class="ai-chat__send"
    >
      <i class="fas fa-paper-plane"></i>
    </button>
  </div>
</div>
</div>
</template>
```

7.2 Componente AIChat - Script (components/common/AIChat.vue)

Arquivo: src/components/common/AIChat.vue (linhas 90-150)

```
<script>
import { geminiService } from '@services/geminiService';

export default {
  name: 'AIChat',
  data() {
    return {
      isExpanded: false,
      hasInteracted: false,
      messages: [
        {
          type: 'assistant',
          text: 'Olá! Sou o QualiBot 2.0, especialista em licitações. Como posso ajudar?'
        }
      ],
      currentMessage: '',
      isLoading: false
    }
  },
  methods: {
    toggleChat() {
      this.isExpanded = !this.isExpanded
      if (this.isExpanded) {
        this.hasInteracted = true
        this.$nextTick(() => {
          this.scrollToBottom()
        })
      }
    },
    async sendMessage() {
      if (!this.currentMessage.trim() || this.isLoading) return

      const userMessage = this.currentMessage.trim()

      // Adicionar mensagem do usuário
      this.messages.push({
        type: 'user',
        text: userMessage
      })

      this.currentMessage = ''
      this.isLoading = true
      this.scrollToBottom()

      try {
        // Chamar serviço Gemini com fallback automático
        const response = await geminiService.chat(userMessage)

        // Adicionar resposta do assistente
        this.messages.push({
          type: 'assistant',
          text: response
        })
      } catch (error) {
        console.error('Erro ao enviar mensagem:', error)
        this.messages.push({
          type: 'assistant',

```

```
        text: error.message || 'Desculpe, ocorreu um erro. Tente novamente.'
      })
    } finally {
      this.isLoading = false
      this.scrollToBottom()
    }
  },

  scrollToBottom() {
    this.$nextTick(() => {
      const container = this.$refs.messagesContainer
      if (container) {
        container.scrollTop = container.scrollHeight
      }
    })
  }
}
</script>
```

8. OBSERVAÇÕES FINAIS

SOBRE ESTE DOCUMENTO

Este documento contém extratos não sequenciais do código-fonte do sistema "Comprar Bem", conforme permitido pelo Art. 2º da Instrução Normativa INPI/PR nº 95/2018.

Arquivos incluídos neste documento:

- package.json - Configuração e dependências do projeto
- src/main.js - Ponto de entrada da aplicação
- src/store/index.js - Gerenciamento de estado global
- src/router/index.js - Sistema de roteamento e autenticação
- src/services/supabase.js - Integração com banco de dados
- src/services/geminiService.js - Sistema de IA com fallback automático
- src/services/processosAdministrativosService.js - Gestão de processos
- src/services/produtoService.js - Gestão de produtos e notificações
- src/components/common/AIChat.vue - Interface do assistente de IA

Características técnicas distintivas:

- Sistema multi-tenant com isolamento de dados por tenant_id
- Arquitetura de fallback automático entre 4 modelos de IA (Gemini 2.0, 1.5, 1.0)
- Sistema de retry com exponential backoff para resiliência
- Controle de acesso baseado em perfis (CPM, CCL, Órgão Admin, Assessoria)
- Notificações automáticas por email integradas ao fluxo de trabalho
- Assistente de IA especializado em legislação de licitações públicas (QualiBot 2.0)

Total de linhas de código apresentadas: Aproximadamente 900 linhas

Arquitetura: Single Page Application (SPA) com Vue.js 2.7

Paradigma: Component-based architecture com padrão Store (Vuex)

COMPRAR BEM - Sistema de Gestão de Compras Públicas

Documento gerado em: Dezembro de 2025

Finalidade: Registro de Software no INPI